

- How do you design good algorithms?
 - Creativity, and
 - Perseverance !

Data Structures (ds)?

- ds is information stored in a format!
- e.g. array is a ds. [helped in ItFib(n)]
 - ↳ Sorted array is an even better ds, Why?

Example 4: Directory / Dictionary

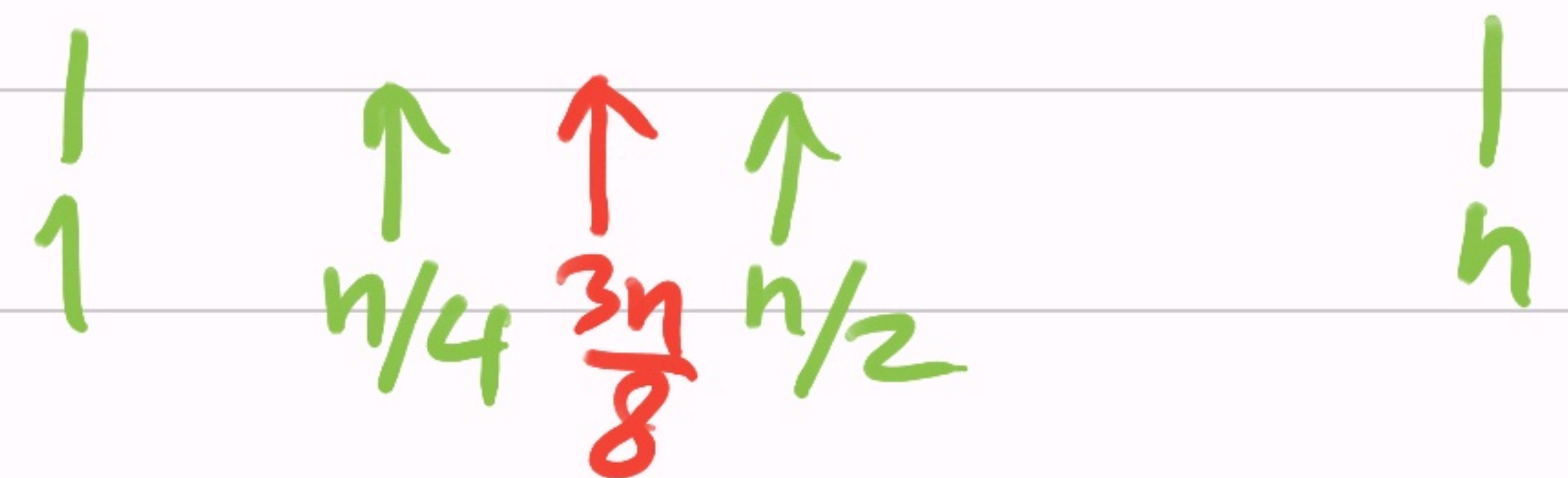
- Suppose it has $n = 10^8$ words.

WAP: $\left\{ \begin{array}{l} \text{Input: Dictionary } A \text{ \& string } s. \\ \text{Output: Yes iff } s \in A. \end{array} \right.$

Ideas? (i) Sequentially search s in A .
 $\triangleright \# \text{ steps} \approx n \approx 10^8 \times 10^{-9} \text{ s} \approx 0.1 \text{ s/query}.$

(ii) Binary Search for s in sorted A .

$\triangleright \# \text{ steps} \approx \lg n \approx 24 \times 1 \text{ ns}$
 $\approx 30 \text{ ns/query}.$



$\Rightarrow$ DS stores data in the memory in a way that each subsequent operation (query / update) is fast & a ds-invariant is maintained.

— Sorting & ds are used heavily in all applications.

Problem: Range-Minima.

WAP: { Input: Array A of n integers.
 { Output: A fast ds to answer queries of the type: Rng-Min(i, j) finds the min of $A[i], A[i+1], \dots, A[j]$.

- eg. $A = [12, 3, 46, 34, 115]$

$\Rightarrow \text{Rng-Min}(i, j) = 34.$

$\uparrow$
 $i=2$ $\uparrow$
 $j=4$

- Applied in: Computational Geometry / String Matching / other algorithms.

Ideas?: (i) Sort A in $n \log n$ time.
 $\Rightarrow$ each query takes $\# \text{steps} = 1$.
but, sorting may not be allowed.

(ii) Rng-Min-brute(i, j): Sequentially search for min in the interval. $\# \text{steps} \approx n$.

(iii) $Rng-Min-Mat(i,j)$: Store $Rng-Min(i,j)$ in the (i,j) -th entry of an $n \times n$ matrix M .
↳ M requires a 1-time cost.

▷ time & HDD-space $\approx n^2 \approx 10^{16} \times 1ns$
↳ worse than before.

Qn: Is there a ds that solves $Rng-Min$ in efficient time & space?

— This course covers many: { algo, paradigms & ds, designs }
& almost-rigorous analysis.

— We saw IterFib(n) takes n steps and the same space.

Qn: Find a superfast algo. for $F(n) \bmod 2024$
 m !!?

— Ideas? (i) IterFib(n, m): for ($i = 2$ to n)
 $F[i] = F[i-1] + F[i-2]$
 $\bmod m$;

▷ IterFib(n, m) requires $> 3n$ instructions & only
(linear but bad) $n \cdot \lg m$ space.

(ii) RecFib(n, m): if ($n \geq 2$) return RecFib($n-1, m$) +
" ($n-2, m$) $\bmod m$
else return n ;

▷ RecFib(n,m) requires $> F(n)$ steps $> 1.5^{n-1}$
[exp. time]

(iii) Clever insight: Do double instead of ± 1 !

$$\begin{pmatrix} F(i) \\ F(i-1) \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix} \cdot \begin{pmatrix} F(i-1) \\ F(i-2) \end{pmatrix}$$

[Evolution of F as a
2x2 matrix]

$$A := \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}$$

$$= A^2 \cdot \begin{pmatrix} F(i-2) \\ F(i-3) \end{pmatrix}$$

$$= \dots = A^{i-1} \cdot \begin{pmatrix} F(1) \\ F(0) \end{pmatrix} = A^{i-1} \cdot \begin{pmatrix} 1 \\ 0 \end{pmatrix}$$

$$\triangleright F(n) = (A^{n-1})_{1,1}.$$

$$\triangleright F(n) \equiv (A^{n-1} \bmod m)_{1,1}.$$

Insight of squaring: Don't multiply when you can square!

-1. $A, A^2, A^4, A^8, A^{16}, \dots, A^k \bmod m$,
[Repeated squaring]

▷ Get to 2^k -power in k steps.

▷ Get to $(n-1)$ th in $\lceil \lg n \rceil$ steps.

▷ Computing $B \times C \bmod m$ takes $(4^2 + 4)$ instr.
 $\Rightarrow$ " $20 \lg m$ steps.

▷ $A^{2^k} \bmod m$ takes $20k \lg m$ steps.
[Exercise] (& $4 \lg m$ space)

- Clever Fib(n, m) :

- $S[0] \leftarrow A := \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}$; // A^{2^0}
- $k \leftarrow \lceil \lg(n-1) \rceil$;
- Write $n-1 \rightarrow b_0 + b_1 \cdot 2 + b_2 \cdot 2^2 + \dots + b_{k-1} \cdot 2^{k-1}$
// $n-1$ in binary

2 for loops $\rightarrow$ • for ($i = 1$ to $k-1$)

$S[i] \leftarrow S[i-1]^2 \bmod m$ // Squaring

$\rightarrow$ • $B \leftarrow S[0]^{b_0} \times S[1]^{b_1} \times \dots \times S[k-1]^{b_{k-1}} \bmod m$

// $B = A^{b_0 + 2b_1 + \dots + 2^{k-1}b_{k-1}} = A^{n-1} \bmod m$

• return B ; // $B_{1,1} = F(n) \bmod m$